02. L1 ophost 모듈 상세 분석

개요

ophost 모듈은 OPinit 시스템의 L1(호스트 체인) 측 핵심 컴포넌트로, L2 롤업 체인과의 브리지 기능을 담당합니다. 이 모듈은 토큰 입금/출금, output root 관리, 브리지 설정 등의 핵심 기능을 제공하며, CosmosSDK의 collections 패키지를 활용한 효율적인 상태 관리를 구현합니다.

Keeper 구조 분석

기본 구조

`x/ophost/keeper/keeper.go` 에 정의된 Keeper 구조체는 ophost 모듈의 핵심 상태 관리자입니다:

```
type Keeper struct {
cdc codec.Codec
storeService corestoretypes.KVStoreService
msgRouter baseapp.MessageRouter

authKeeper types.AccountKeeper
bankKeeper types.BankKeeper
bridgeHook types.BridgeHook
communityPoolKeeper types.CommunityPoolKeeper

authority  string
```

```
Schema collections.Schema

NextBridgeId collections.Sequence

Params collections.Item[types.Params]

BridgeConfigs collections.Map[uint64, types.BridgeConfig]

BatchInfos collections.Map[collections.Pair[uint64, uint64], types.BatchInfoWith

NextL1Sequences collections.Map[uint64, uint64]

TokenPairs collections.Map[collections.Pair[uint64, string], string]

OutputProposals collections.Map[collections.Pair[uint64, uint64], types.Output]

NextOutputIndexes collections.Map[uint64, uint64]

ProvenWithdrawals collections.Map[collections.Pair[uint64, []byte], bool]

MigrationInfos collections.Map[collections.Pair[uint64, string], types.Migration

}
```

주요 컴포넌트 설명

1. 기본 의존성

- **cdc**: 코덱으로 데이터 직렬화/역직렬화 담당
- **storeService**: KV 스토어 서비스로 상태 저장소 접근
- **msgRouter**: 메시지 라우터로 트랜잭션 처리
- **authority**: 거버넌스 권한을 가진 주소 (일반적으로 x/gov 모듈)

2. 외부 Keeper 의존성

- **authKeeper**: 계정 관리 (브리지 계정 생성 등)

- **bankKeeper**: 토큰 전송 및 잔액 관리
- **communityPoolKeeper**: 커뮤니티 풀 관리 (등록 수수료 처리)
- **bridgeHook**: 브리지 이벤트 후킹 메커니즘

3. Collections 기반 상태 관리

ophost 모듈은 CosmosSDK의 collections 패키지를 활용하여 효율적인 상태 관리를 구현합니다:

- **NextBridgeId**: 다음 브리지 ID 시퀀스
- **BridgeConfigs**: 브리지별 설정 정보 저장
- **BatchInfos**: 배치 정보와 output 데이터 저장
- **NextL1Sequences**: 브리지별 다음 L1 시퀀스 번호
- **TokenPairs**: 브리지별 토큰 페어 매핑
- **OutputProposals**: output root 제안 저장
- **NextOutputIndexes**: 브리지별 다음 output 인덱스
- **ProvenWithdrawals**: 검증된 출금 트랜잭션 기록
- **MigrationInfos**: 토큰 마이그레이션 정보

주요 메시지 타입 분석

1. MsgInitiateTokenDeposit

L1에서 L2로 토큰을 입금하는 메시지입니다.

```
type MsgInitiateTokenDeposit struct {
Sender  string
To  string
```

```
Amount sdk.Coin

BridgeId  uint64

Data []byte

}
```

주요 필드

- **Sender**: L1에서 토큰을 보내는 주소
- **To**: L2에서 토큰을 받을 주소 (다양한 형식 지원)
- **Amount**: 전송할 토큰 수량 (0도 허용 - 계정 생성용)
- **BridgeId**: 사용할 브리지 ID
- **Data**: 추가 데이터 (최대 10KB)

검증 로직

```
func (msg MsgInitiateTokenDeposit) Validate(ac address.Codec) error {

// Sender 주소 검증

if  _, err := ac.StringToBytes(msg.Sender);  err != nil {

return  err

}


// To 주소 비어있지 않은지 확인 (형식은 체인별로 다를 수 있음)

if  len(msg.To) == 0 {

return  sdkerrors.ErrInvalidAddress.Wrap("to address cannot be empty")

}
```

```go
// 토큰 수량 유효성 검사 (0 허용)
if !msg.Amount.IsValid() {
return  ErrInvalidAmount
}

// 브리지 ID 검증
if  msg.BridgeId == 0 {
return  ErrInvalidBridgeId
}

// 데이터 크기 제한 확인
if  len(msg.Data) > MaxDataLength {
return  ErrInvalidData.Wrapf("data length exceeds %d", MaxDataLength)
}

return  nil
}
```

2. MsgFinalizeTokenWithdrawal

L2에서 L1으로의 토큰 출금을 완료하는 메시지입니다.

```go
type MsgFinalizeTokenWithdrawal struct {
```

```
Sender  string

BridgeId  uint64

OutputIndex  uint64

WithdrawalProofs [][]byte

From  string

To  string

Sequence  uint64

Amount sdk.Coin

Version []byte

StorageRoot []byte

LastBlockHash []byte

}
```

주요 필드

- **Sender**: 출금을 실행하는 L1 주소
- **BridgeId**: 브리지 ID
- **OutputIndex**: 참조하는 output의 인덱스
- **WithdrawalProofs**: Merkle proof 배열 (각각 32바이트)
- **From**: L2에서 토큰을 보낸 주소
- **To**: L1에서 토큰을 받을 주소
- **Sequence**: L2 출금 시퀀스 번호
- **Amount**: 출금할 토큰 수량
- **Version**: 버전 정보 (1바이트)

- **StorageRoot**: 스토리지 루트 해시 (32바이트)
- **LastBlockHash**: 마지막 블록 해시 (32바이트)

검증 로직

```
func (msg MsgFinalizeTokenWithdrawal) Validate(ac address.Codec) error {
// 각 withdrawal proof가 32바이트인지 확인
for _, proof := range msg.WithdrawalProofs {
if len(proof) != 32 {
return ErrInvalidHashLength.Wrap("withdrawal_proofs")
}
}

// 버전이 1바이트인지 확인
if len(msg.Version) != 1 {
return ErrInvalidHashLength.Wrap("version")
}

// 스토리지 루트가 32바이트인지 확인
if len(msg.StorageRoot) != 32 {
return ErrInvalidHashLength.Wrap("storage_root")
}

// 마지막 블록 해시가 32바이트인지 확인
```

```
if len(msg.LastBlockHash) != 32 {
return ErrInvalidHashLength.Wrap("last_block_hash")
}

return nil
}
```

3. MsgProposeOutput

L2 output root를 제안하는 메시지입니다.

```
type MsgProposeOutput struct {
Proposer string
BridgeId uint64
OutputIndex uint64
L2BlockNumber uint64
OutputRoot []byte
}
```

주요 필드

- **Proposer**: output을 제안하는 주소 (브리지 설정의 proposer와 일치해야 함)
- **BridgeId**: 브리지 ID
- **OutputIndex**: output 인덱스
- **L2BlockNumber**: L2 블록 번호

- **OutputRoot**: output root 해시 (32바이트)

4. MsgDeleteOutput

잘못된 output을 삭제하는 메시지입니다 (challenger 전용).

```
type MsgDeleteOutput struct {
Challenger  string
BridgeId  uint64
OutputIndex  uint64
}
```

5. MsgCreateBridge

새로운 브리지를 생성하는 메시지입니다.

```
type MsgCreateBridge struct {
Creator  string
Config BridgeConfig
}
```

핵심 데이터 구조 분석

1. BridgeConfig

브리지의 핵심 설정을 담는 구조체입니다.

```go
type BridgeConfig struct {
Challenger  string
Proposer  string
SubmissionInterval time.Duration
FinalizationPeriod time.Duration
SubmissionStartHeight  uint64
BatchInfo BatchInfo
Metadata []byte
OracleEnabled  bool
}
```

주요 필드 설명

- **Challenger**: 잘못된 output을 챌린지할 수 있는 주소
- **Proposer**: output root를 제안할 수 있는 주소
- **SubmissionInterval**: output 제출 간격
- **FinalizationPeriod**: output 확정 기간 (챌린지 기간)
- **SubmissionStartHeight**: 제출 시작 높이
- **BatchInfo**: 배치 제출자 정보
- **Metadata**: 브리지 메타데이터 (최대 5KB)
- **OracleEnabled**: 오라클 활성화 여부

검증 로직

```go
func (config BridgeConfig) Validate(ac address.Codec) error {
```

```go
// Challenger와 Proposer 주소 검증
if _, err := ac.StringToBytes(config.Challenger); err != nil {
return err
}
if _, err := ac.StringToBytes(config.Proposer); err != nil {
return err
}

// BatchInfo 검증
if config.BatchInfo.ChainType == BatchInfo_UNSPECIFIED {
return errors.Wrapf(sdkerrors.ErrInvalidRequest, "batch chain type must be set"
}
if config.BatchInfo.Submitter == "" {
return errors.Wrapf(sdkerrors.ErrInvalidRequest, "batch submitter must be set")
}

// 시간 설정 검증
if config.FinalizationPeriod == time.Duration(0) {
return errors.Wrapf(sdkerrors.ErrInvalidRequest, "finalization period must be g
}
if config.SubmissionInterval == time.Duration(0) {
return errors.Wrapf(sdkerrors.ErrInvalidRequest, "submission interval must be g
}
```

```go
// 시작 높이 검증
if  config.SubmissionStartHeight == 0 {
return  errors.Wrapf(sdkerrors.ErrInvalidRequest, "submission start height must
}

return  nil
}
```

2. Output

L2 상태의 output root를 나타내는 구조체입니다.

```go
type Output struct {
OutputRoot []byte
L1BlockNumber  uint64
L2BlockNumber  uint64
L1Timestamp time.Time
}
```

주요 필드 설명

- **OutputRoot**: L2 상태의 Merkle root (32바이트)
- **L1BlockNumber**: 해당 L1 블록 번호

- **L2BlockNumber**: 해당 L2 블록 번호
- **L1Timestamp**: L1 타임스탬프

Output Root 생성 함수

```
func GenerateOutputRoot(version byte, storageRoot []byte, latestBlockHash []by
seed := make([]byte, 1+32+32)
seed[0] = version
copy(seed[1:], storageRoot[:32])
copy(seed[1+32:], latestBlockHash[:32])
return sha3.Sum256(seed)
}
```

3. BatchInfo

배치 제출자 정보를 담는 구조체입니다.

```
type BatchInfo struct {
Submitter string
ChainType BatchInfo_ChainType
}
```

ChainType 열거형

- **UNSPECIFIED**: 미지정 (유효하지 않음)
- **INITIA**: Initia 체인

- **ETHEREUM**: 이더리움 체인

핵심 함수 동작 분석

1. 브리지 생성 프로세스

`CreateBridge` 메시지 처리 과정:

1. **등록 수수료 처리**: 설정된 등록 수수료를 커뮤니티 풀로 전송
2. **브리지 ID 생성**: `IncreaseNextBridgeId()` 로 새로운 브리지 ID 할당
3. **브리지 설정 저장**: `SetBridgeConfig()` 로 브리지 설정 저장
4. **배치 정보 초기화**: `SetBatchInfo()` 로 배치 정보 초기화
5. **브리지 계정 생성**: 브리지 전용 계정 생성 및 등록

2. 토큰 입금 프로세스

`InitiateTokenDeposit` 메시지 처리:

1. **메시지 검증**: 주소, 수량, 브리지 ID 등 검증
2. **L1 시퀀스 증가**: `IncreaseNextL1Sequence()` 로 시퀀스 번호 할당
3. **토큰 전송**: 사용자 → 브리지 계정으로 토큰 전송
4. **이벤트 발생**: 입금 이벤트 발생으로 L2에서 감지 가능

3. 토큰 출금 완료 프로세스

`FinalizeTokenWithdrawal` 메시지 처리:

1. **Output 검증**: 해당 output이 존재하고 확정되었는지 확인
2. **Merkle Proof 검증**: withdrawal proof를 통한 출금 유효성 검증
3. **중복 출금 방지**: `ProvenWithdrawals` 에서 이미 처리된 출금인지 확인

4. **토큰 전송**: 브리지 계정 → 수신자로 토큰 전송

5. **출금 기록**: `RecordProvenWithdrawal()` 로 출금 완료 기록

4. Output 제안 프로세스

`ProposeOutput` 메시지 처리:

1. **권한 검증**: 메시지 발신자가 설정된 proposer인지 확인

2. **Output 인덱스 검증**: 올바른 순서의 output인지 확인

3. **Output 저장**: `OutputProposals` 에 output 정보 저장

4. **인덱스 증가**: `NextOutputIndexes` 업데이트

에러 처리 및 보안

주요 에러 타입

```
var (
ErrInvalidBridgeId = errorsmod.Register(ModuleName, 2, "invalid bridge id")
ErrInvalidHashLength = errorsmod.Register(ModuleName, 3, "invalid hash length")
ErrInvalidOutputIndex = errorsmod.Register(ModuleName, 4, "invalid output index"
ErrInvalidAmount = errorsmod.Register(ModuleName, 5, "invalid bridge amount")
ErrInvalidSequence = errorsmod.Register(ModuleName, 6, "invalid sequence")
ErrNotFinalized = errorsmod.Register(ModuleName, 8, "output has not finalized")
ErrAlreadyFinalized = errorsmod.Register(ModuleName, 9, "output has already fina
ErrFailedToVerifyWithdrawal = errorsmod.Register(ModuleName, 10, "failed to veri
ErrWithdrawalAlreadyFinalized = errorsmod.Register(ModuleName, 11, "withdrawal a
```

)

보안 메커니즘

1. **권한 기반 접근 제어**: proposer와 challenger 역할 분리
2. **Finalization Period**: output 확정 전 챌린지 기간 제공
3. **Merkle Proof 검증**: 출금 시 암호학적 증명 요구
4. **중복 처리 방지**: 동일한 출금의 중복 실행 방지
5. **주소 검증**: 모든 주소 형식의 유효성 검사

상태 관리 최적화

ophost 모듈은 CosmosSDK의 collections 패키지를 활용하여 효율적인 상태 관리를 구현합니다:

- **인덱싱**: 복합 키를 통한 빠른 조회
- **타입 안전성**: 컴파일 타임 타입 검사
- **메모리 효율성**: 지연 로딩 및 캐싱
- **반복자 지원**: 효율적인 범위 쿼리

이러한 설계를 통해 ophost 모듈은 L1에서 안전하고 효율적인 브리지 기능을 제공하며, L2 롤업 체인과의 신뢰할 수 있는 상호작용을 보장합니다.